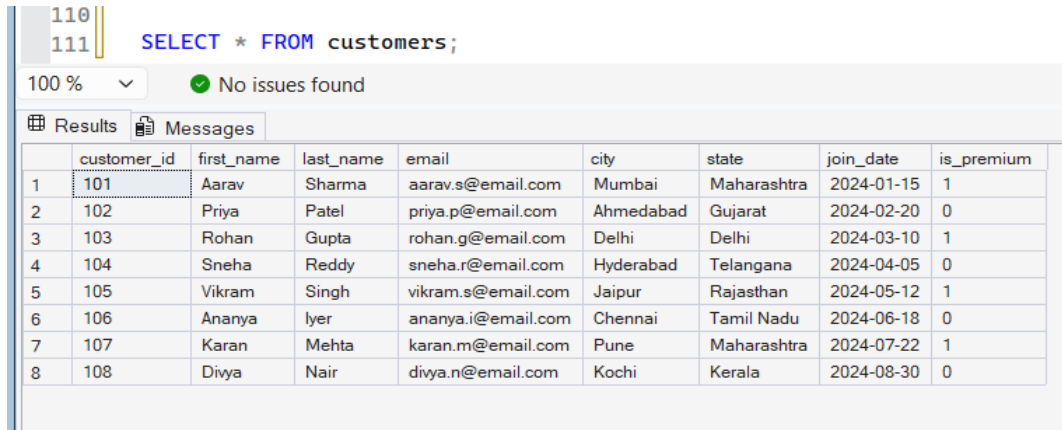


SECTION – A

1. Display all columns and rows from customers table

SELECT * FROM customers;



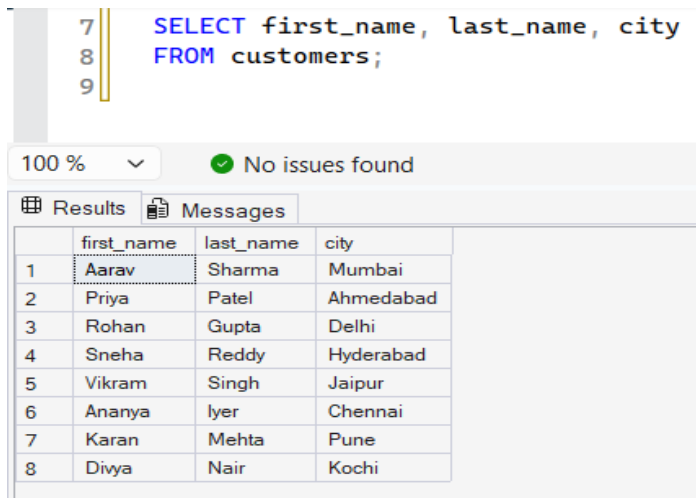
The screenshot shows a SQL query execution interface. At the top, the query `SELECT * FROM customers;` is entered. Below the query, a status bar indicates '100 %' and 'No issues found'. The results are displayed in a table with 9 columns: `customer_id`, `first_name`, `last_name`, `email`, `city`, `state`, `join_date`, and `is_premium`. The table contains 8 rows of data.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

2. Retrieve first_name, last_name and city

SELECT first_name, last_name, city

FROM customers;

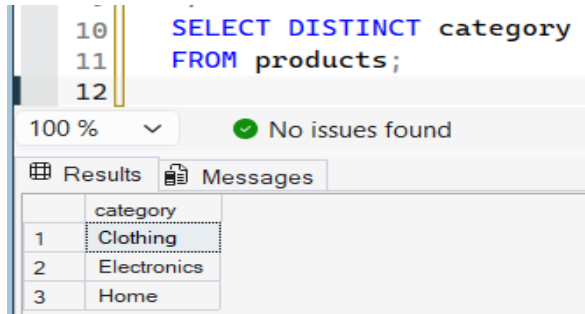


The screenshot shows a SQL query execution interface. At the top, the query `SELECT first_name, last_name, city FROM customers;` is entered. Below the query, a status bar indicates '100 %' and 'No issues found'. The results are displayed in a table with 4 columns: `first_name`, `last_name`, and `city`. The table contains 8 rows of data.

	first_name	last_name	city
1	Aarav	Sharma	Mumbai
2	Priya	Patel	Ahmedabad
3	Rohan	Gupta	Delhi
4	Sneha	Reddy	Hyderabad
5	Vikram	Singh	Jaipur
6	Ananya	Iyer	Chennai
7	Karan	Mehta	Pune
8	Divya	Nair	Kochi

3. List unique categories in products table

```
SELECT DISTINCT category  
FROM products;
```



4. Identify the Primary Keys of each table. Why Primary Key must be UNIQUE and NOT NULL?

TABLES	PRIMARY KEY
customers	customer_id
orders	order_id
products	product_id
order_items	item_id

Why Primary Key must be UNIQUE and NOT NULL?

- Ensures each record is uniquely identified.
- Prevents duplicate records.
- NULL values are not allowed because every row must have an identifier.

**5. What constraints are applied to the email column in the customers table?
What would happen if you tried to insert a duplicate email?**

email VARCHAR(100) UNIQUE NOT NULL

Constraints:

- UNIQUE → No duplicate emails allowed.
- NOT NULL → Email cannot be empty.

6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

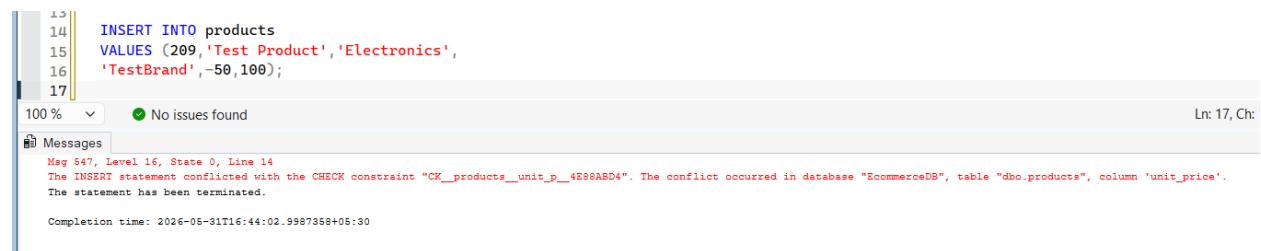
INSERT INTO products

VALUES (209,'Test Product','Electronics',

'TestBrand',-50,100);

Error occurs because negative values Violate the CHECK constraint:

CHECK (unit_price > 0)



```
13 |  
14 | INSERT INTO products  
15 | VALUES (209,'Test Product','Electronics',  
16 | 'TestBrand',-50,100);  
17 |
```

100 % No issues found Ln: 17, Ch:

Messages

Msg 547, Level 16, State 0, Line 14
The INSERT statement conflicted with the CHECK constraint "CK_products__unit_p__4E88ABD4". The conflict occurred in database "EcommerceDB", table "dbo.products", column 'unit_price'.
The statement has been terminated.

Completion time: 2026-05-31T16:44:02.9987358+05:30

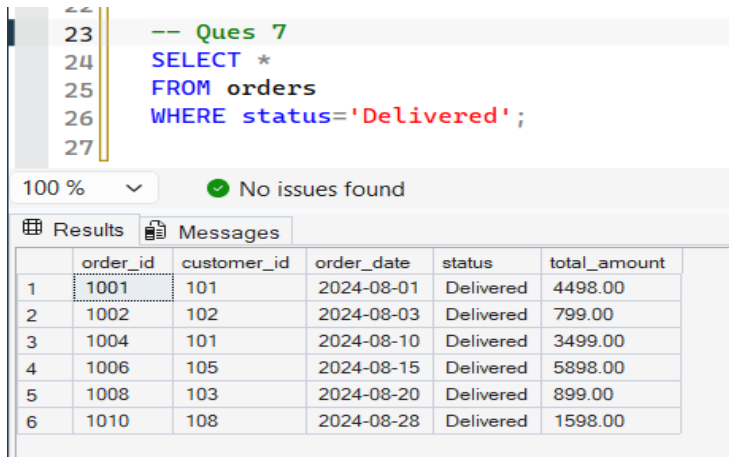
SECTION – B

7. Retrieve all orders with status = 'Delivered'.

SELECT *

FROM orders

WHERE status='Delivered';



The screenshot shows a SQL query editor with the following code:

```
-- Ques 7
SELECT *
FROM orders
WHERE status='Delivered';
```

Below the editor, the results are displayed in a table with 6 rows and 6 columns. The columns are: order_id, customer_id, order_date, status, and total_amount. The status for all rows is 'Delivered'.

	order_id	customer_id	order_date	status	total_amount
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1004	101	2024-08-10	Delivered	3499.00
4	1006	105	2024-08-15	Delivered	5898.00
5	1008	103	2024-08-20	Delivered	899.00
6	1010	108	2024-08-28	Delivered	1598.00

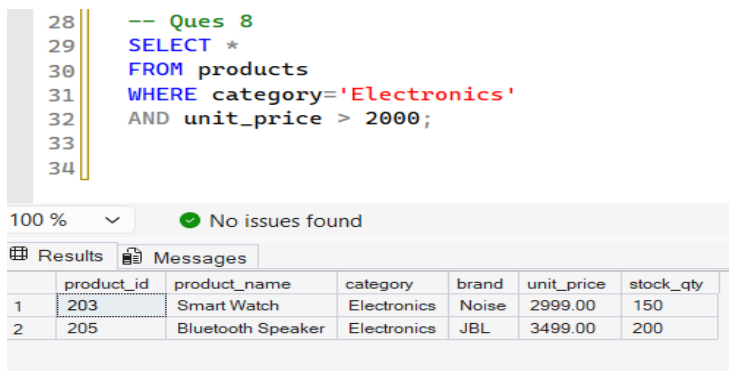
8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SELECT *

FROM products

WHERE category='Electronics'

AND unit_price > 2000;



The screenshot shows a SQL query editor with the following code:

```
-- Ques 8
SELECT *
FROM products
WHERE category='Electronics'
AND unit_price > 2000;
```

Below the editor, the results are displayed in a table with 2 rows and 6 columns. The columns are: product_id, product_name, category, brand, unit_price, and stock_qty. Both products are in the 'Electronics' category and have a unit_price greater than 2000.

	product_id	product_name	category	brand	unit_price	stock_qty
1	203	Smart Watch	Electronics	Noise	2999.00	150
2	205	Bluetooth Speaker	Electronics	JBL	3499.00	200

9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
SELECT *  
  
FROM customers  
  
WHERE state='Maharashtra'  
  
AND join_date BETWEEN '2024-01-01'  
  
AND '2024-12-31';
```

```
35  -- Ques 9  
36  SELECT *  
37  FROM customers  
38  WHERE state='Maharashtra'  
39  AND join_date BETWEEN '2024-01-01'  
40  AND '2024-12-31';  
41
```

100 % No issues found

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
SELECT *  
  
FROM orders  
  
WHERE order_date BETWEEN '2024-08-10'  
  
AND '2024-08-25'  
  
AND status <> 'Cancelled';
```

```
43  -- Ques 10  
44  SELECT *  
45  FROM orders  
46  WHERE order_date BETWEEN '2024-08-10'  
47  AND '2024-08-25'  
48  AND status <> 'Cancelled';  
49
```

100 % No issues found

	order_id	customer_id	order_date	status	total_amount
1	1004	101	2024-08-10	Delivered	3499.00
2	1006	105	2024-08-15	Delivered	5898.00
3	1007	106	2024-08-18	Pending	1299.00
4	1008	103	2024-08-20	Delivered	899.00
5	1009	107	2024-08-25	Shipped	6098.00

11.Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

It speeds up searches and filtering on order_date.

```
CREATE INDEX idx_orders_date
```

```
ON orders(order_date);
```

Example:

```
SELECT *
```

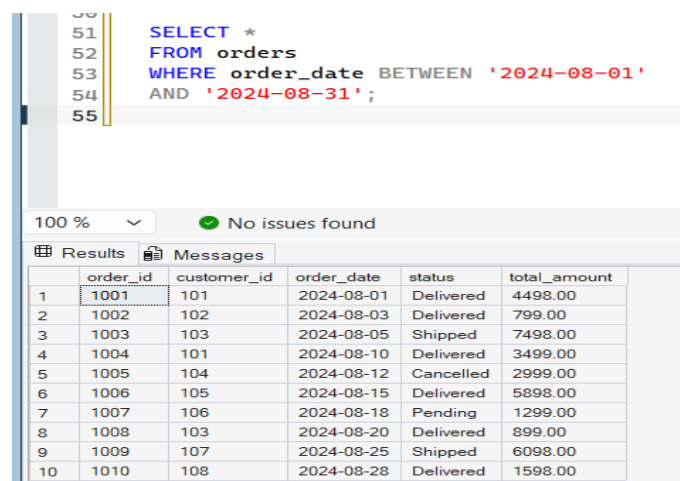
```
FROM orders
```

```
WHERE order_date BETWEEN '2024-08-01'
```

```
AND '2024-08-31';
```

Without index → Full table scan.

With index → Faster lookup using indexed values.



The screenshot shows a SQL IDE interface. At the top, a query is entered in a text editor:

```
51 SELECT *
52 FROM orders
53 WHERE order_date BETWEEN '2024-08-01'
54 AND '2024-08-31';
55
```

Below the editor, the results of the query are displayed in a table. The table has columns: order_id, customer_id, order_date, status, and total_amount. The results are as follows:

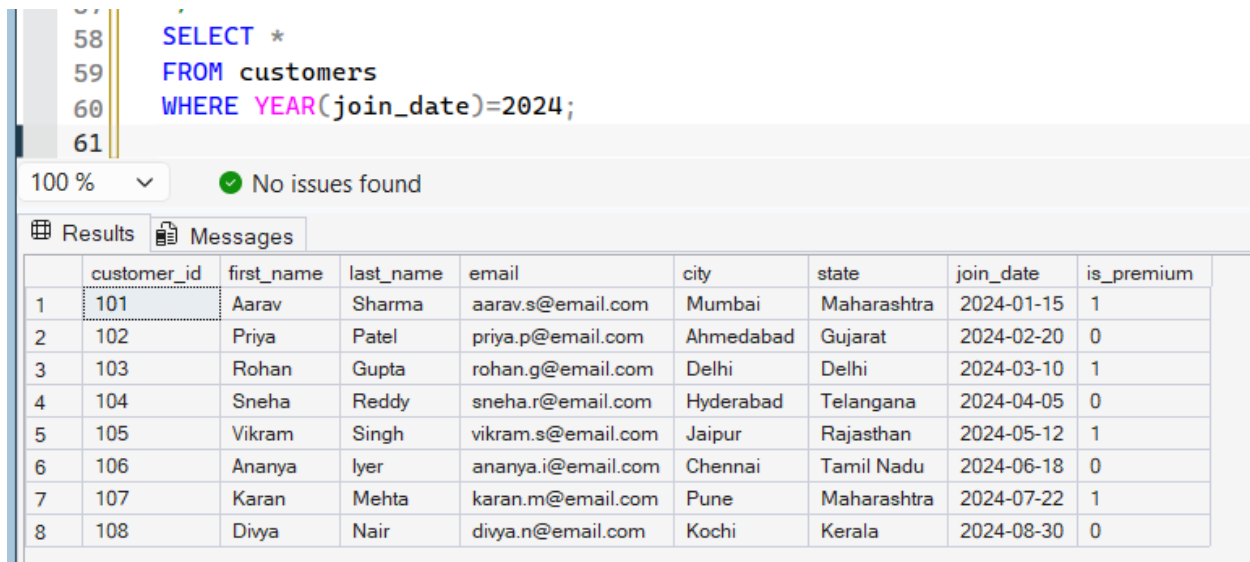
	order_id	customer_id	order_date	status	total_amount
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1003	103	2024-08-05	Shipped	7498.00
4	1004	101	2024-08-10	Delivered	3499.00
5	1005	104	2024-08-12	Cancelled	2999.00
6	1006	105	2024-08-15	Delivered	5898.00
7	1007	106	2024-08-18	Pending	1299.00
8	1008	103	2024-08-20	Delivered	899.00
9	1009	107	2024-08-25	Shipped	6098.00
10	1010	108	2024-08-28	Delivered	1598.00

12.If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

```
SELECT *
```

```
FROM customers
```

```
WHERE YEAR(join_date)=2024;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
58 SELECT *
59 FROM customers
60 WHERE YEAR(join_date)=2024;
61
```

Below the query editor, there is a status bar indicating "100 % No issues found". The results pane shows a table with 9 columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The table contains 8 rows of data.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

No, because function YEAR() is applied on the column.

```
SELECT *
```

```
FROM customers
```

```
WHERE join_date >= '2024-01-01'
```

```
AND join_date < '2025-01-01';
```

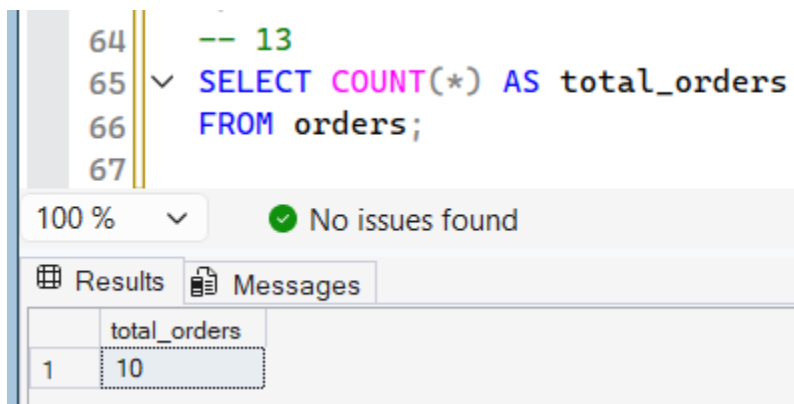
This is called a **SARGable** query.

SECTION – C

13. Count the total number of orders in the orders table.

```
SELECT COUNT(*) AS total_orders
```

```
FROM orders;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
64 -- 13
65 SELECT COUNT(*) AS total_orders
66 FROM orders;
67
```

Below the query editor, there is a status bar indicating "100 %" and "No issues found". The results pane is active, showing a table with one column, "total_orders", and one row with the value "10".

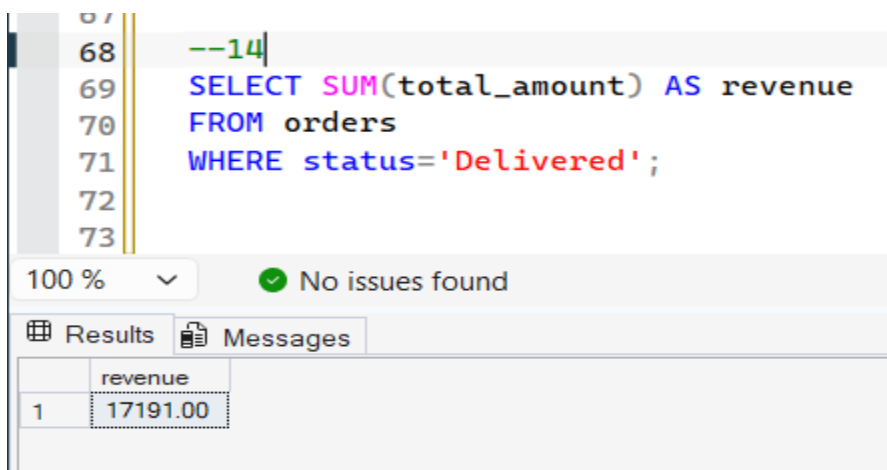
	total_orders
1	10

14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

```
SELECT SUM(total_amount) AS revenue
```

```
FROM orders
```

```
WHERE status='Delivered';
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
67
68 --14
69 SELECT SUM(total_amount) AS revenue
70 FROM orders
71 WHERE status='Delivered';
72
73
```

Below the query editor, there is a status bar indicating "100 %" and "No issues found". The results pane is active, showing a table with one column, "revenue", and one row with the value "17191.00".

	revenue
1	17191.00

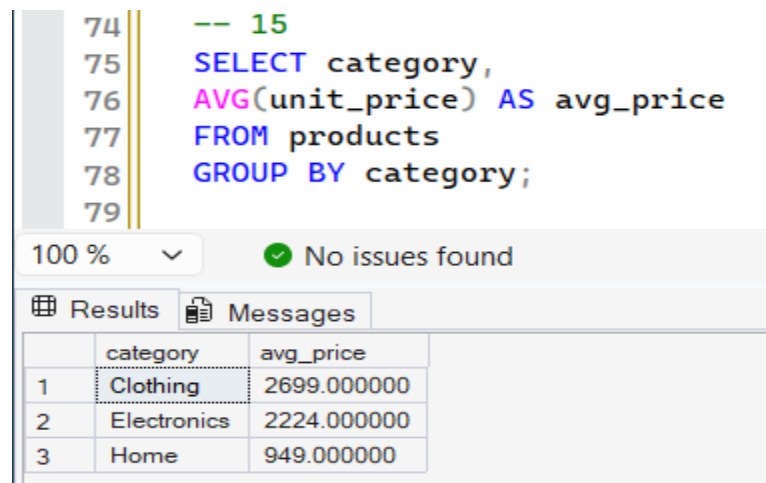
15. Calculate the average unit_price of products in each category.

SELECT category,

AVG(unit_price) AS avg_price

FROM products

GROUP BY category;



The screenshot shows a SQL IDE interface. On the left, line numbers 74 through 79 are visible. The SQL query is as follows:

```
-- 15
SELECT category,
AVG(unit_price) AS avg_price
FROM products
GROUP BY category;
```

Below the query editor, there is a status bar showing "100 %" and "No issues found". Below that, there are tabs for "Results" and "Messages". The "Results" tab is active, displaying a table with the following data:

	category	avg_price
1	Clothing	2699.000000
2	Electronics	2224.000000
3	Home	949.000000

16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

SELECT status,

COUNT(*) AS order_count,

SUM(total_amount) AS total_revenue

FROM orders

GROUP BY status

ORDER BY total_revenue DESC;

```

81  -- 16
82  SELECT status,
83  COUNT(*) AS order_count,
84  SUM(total_amount) AS total_revenue
85  FROM orders
86  GROUP BY status
87  ORDER BY total_revenue DESC;
88

```

100 % No issues found

Results Messages

	status	order_count	total_revenue
1	Delivered	6	17191.00
2	Shipped	2	13596.00
3	Cancelled	1	2999.00
4	Pending	1	1299.00

17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

SELECT category,

MAX(unit_price) AS highest_price,

MIN(unit_price) AS lowest_price

FROM products

GROUP BY category;

```

89  -- 17
90  SELECT category,
91  MAX(unit_price) AS highest_price,
92  MIN(unit_price) AS lowest_price
93  FROM products
94  GROUP BY category;

```

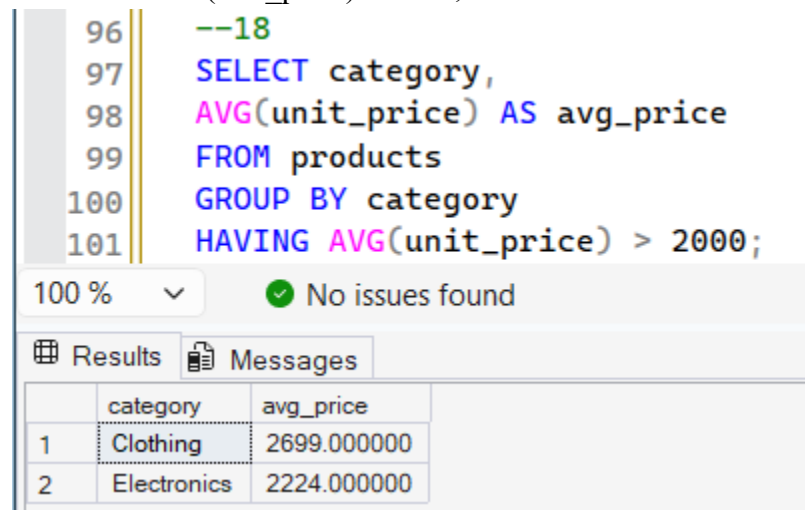
100 % No issues found

Results Messages

	category	highest_price	lowest_price
1	Clothing	4599.00	799.00
2	Electronics	3499.00	899.00
3	Home	1299.00	599.00

18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

```
SELECT category,  
AVG(unit_price) AS avg_price  
FROM products  
GROUP BY category  
HAVING AVG(unit_price) > 2000;
```



The screenshot shows a SQL IDE interface. The top part displays the SQL query being executed. Below the query, there is a status bar indicating '100 %' zoom and 'No issues found'. The bottom part shows the 'Results' tab with a table of query results.

```
96  --18  
97  SELECT category,  
98  AVG(unit_price) AS avg_price  
99  FROM products  
100 GROUP BY category  
101 HAVING AVG(unit_price) > 2000;
```

100 % No issues found

	category	avg_price
1	Clothing	2699.000000
2	Electronics	2224.000000

SECTION – D

19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT o.order_id,  
       o.order_date,  
       c.first_name,  
       c.last_name,  
       o.total_amount  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.customer_id;
```

```
104  --19  
105  SELECT o.order_id,  
106         o.order_date,  
107         c.first_name,  
108         c.last_name,  
109         o.total_amount  
110  FROM orders o  
111  INNER JOIN customers c  
112  ON o.customer_id = c.customer_id;  
113
```

100 % No issues found

Results Messages

	order_id	order_date	first_name	last_name	total_amount
1	1001	2024-08-01	Aarav	Sharma	4498.00
2	1002	2024-08-03	Priya	Patel	799.00
3	1003	2024-08-05	Rohan	Gupta	7498.00
4	1004	2024-08-10	Aarav	Sharma	3499.00
5	1005	2024-08-12	Sneha	Reddy	2999.00
6	1006	2024-08-15	Vikram	Singh	5898.00
7	1007	2024-08-18	Ananya	Iyer	1299.00
8	1008	2024-08-20	Rohan	Gupta	899.00
9	1009	2024-08-25	Karan	Mehta	6098.00
10	1010	2024-08-28	Divya	Nair	1598.00

20. Using a LEFT JOIN, list ALL customers and their orders (if any).

Customers with no orders should still appear with NULL values for order columns.

```
SELECT c.customer_id,  
       c.first_name,  
       c.last_name,  
       o.order_id,  
       o.order_date  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

```
114  -- 20  
115  SELECT c.customer_id,  
116         c.first_name,  
117         c.last_name,  
118         o.order_id,  
119         o.order_date  
120  FROM customers c  
121  LEFT JOIN orders o  
122  ON c.customer_id = o.customer_id;  
123
```

100 % No issues found

Results Messages

	customer_id	first_name	last_name	order_id	order_date
1	101	Aarav	Sharma	1001	2024-08-01
2	101	Aarav	Sharma	1004	2024-08-10
3	102	Priya	Patel	1002	2024-08-03
4	103	Rohan	Gupta	1003	2024-08-05
5	103	Rohan	Gupta	1008	2024-08-20
6	104	Sneha	Reddy	1005	2024-08-12
7	105	Vikram	Singh	1006	2024-08-15
8	106	Ananya	Iyer	1007	2024-08-18
9	107	Karan	Mehta	1009	2024-08-25
10	108	Divya	Nair	1010	2024-08-28

21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```
SELECT o.order_id,  
       p.product_name,  
       oi.quantity,  
       oi.unit_price,  
       oi.discount_pct  
FROM orders o  
JOIN order_items oi  
ON o.order_id = oi.order_id  
JOIN products p  
ON oi.product_id = p.product_id;
```

```
125  --21  
126  SELECT o.order_id,  
127         p.product_name,  
128         oi.quantity,  
129         oi.unit_price,  
130         oi.discount_pct  
131  FROM orders o  
132  JOIN order_items oi  
133  ON o.order_id = oi.order_id  
134  JOIN products p  
135  ON oi.product_id = p.product_id;  
136
```

100 % No issues found

	order_id	product_name	quantity	unit_price	discount_pct
1	1001	Wireless Earbuds	2	1499.00	0.00
2	1001	Laptop Stand	1	899.00	10.00
3	1002	Cotton T-Shirt	1	799.00	0.00
4	1003	Smart Watch	1	2999.00	0.00
5	1003	Running Shoes	1	4599.00	5.00
6	1004	Bluetooth Speaker	1	3499.00	0.00
7	1005	Smart Watch	1	2999.00	0.00
8	1006	Wireless Earbuds	1	1499.00	10.00
9	1006	Running Shoes	1	4599.00	5.00
10	1007	Bedsheet Set	1	1299.00	0.00
11	1008	Laptop Stand	1	899.00	0.00
12	1009	Bluetooth Speaker	1	3499.00	0.00
13	1009	Cushion Covers (Set)	2	599.00	15.00
14	1010	Bedsheet Set	1	1299.00	0.00
15	1010	Cushion Covers (Set)	1	599.00	0.00

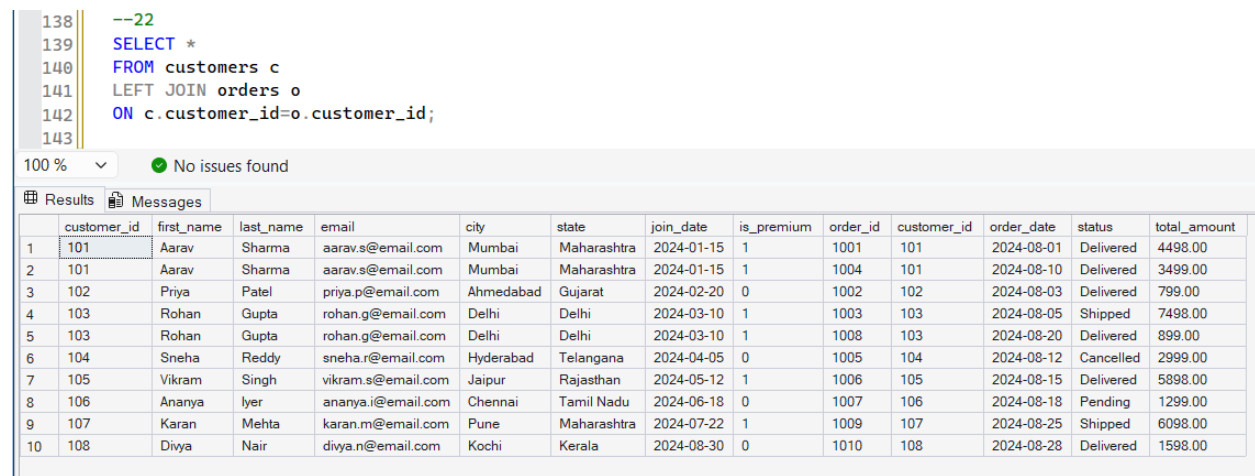
22.Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN : Returns all rows from left table.

Example:

```
SELECT *  
  
FROM customers c  
  
LEFT JOIN orders o  
  
ON c.customer_id=o.customer_id;
```

All Customers Appears.



The screenshot shows a SQL query editor with a query window on the left and a results window on the right. The query window contains the following SQL code:

```
138 --22  
139 SELECT *  
140 FROM customers c  
141 LEFT JOIN orders o  
142 ON c.customer_id=o.customer_id;  
143
```

The results window shows a table with 14 columns: customer_id, first_name, last_name, email, city, state, join_date, is_premium, order_id, customer_id, order_date, status, and total_amount. The table contains 10 rows of data, representing all customers from the customers table, each with their associated order information where available.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
2	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
3	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
4	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
5	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
6	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
7	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
8	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
9	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00
10	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00

RIGHT JOIN

```
SELECT *  
  
FROM customers c  
  
RIGHT JOIN orders o  
  
ON c.customer_id=o.customer_id;
```

All orders appear even if customer missing.

```

144 SELECT *
145 FROM customers c
146 RIGHT JOIN orders o
147 ON c.customer_id=o.customer_id;
148

```

100 % ● No issues found

	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
4	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
5	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
6	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
7	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
8	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
9	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00
10	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00

FULL OUTER JOIN

Returns all rows from both tables.

Useful when finding unmatched records in either table.

23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Foreign Keys:

1.

orders.customer_id

REFERENCES customers(customer_id)

2.

order_items.order_id

REFERENCES orders(order_id)

3.

order_items.product_id

REFERENCES products(product_id)

If you tried to insert an order with customer_id = 999, then error occurs because customer_id 999 does not exist.

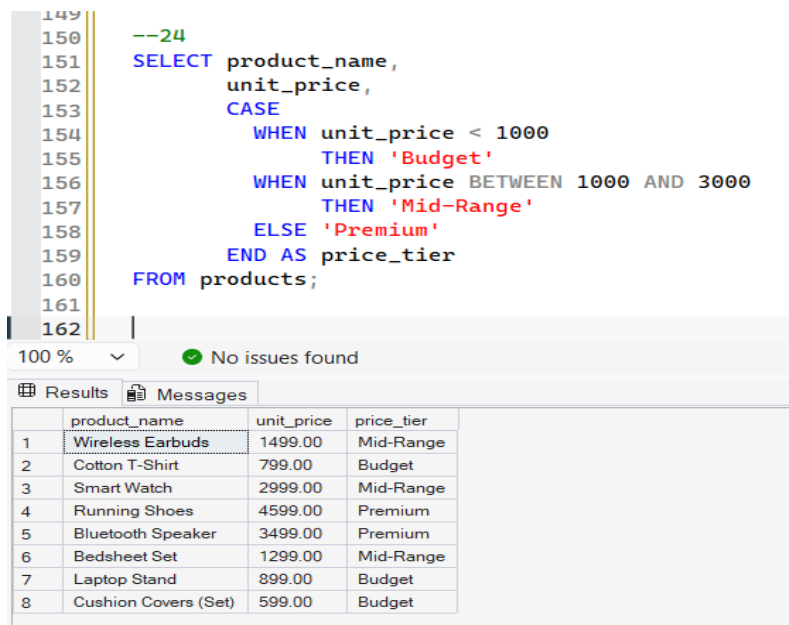
SECTION – E

24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

```
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000  
           THEN 'Budget'  
         WHEN unit_price BETWEEN 1000 AND 3000  
           THEN 'Mid-Range'  
         ELSE 'Premium'  
       END AS price_tier  
FROM products;
```



The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```
--24  
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000  
           THEN 'Budget'  
         WHEN unit_price BETWEEN 1000 AND 3000  
           THEN 'Mid-Range'  
         ELSE 'Premium'  
       END AS price_tier  
FROM products;
```

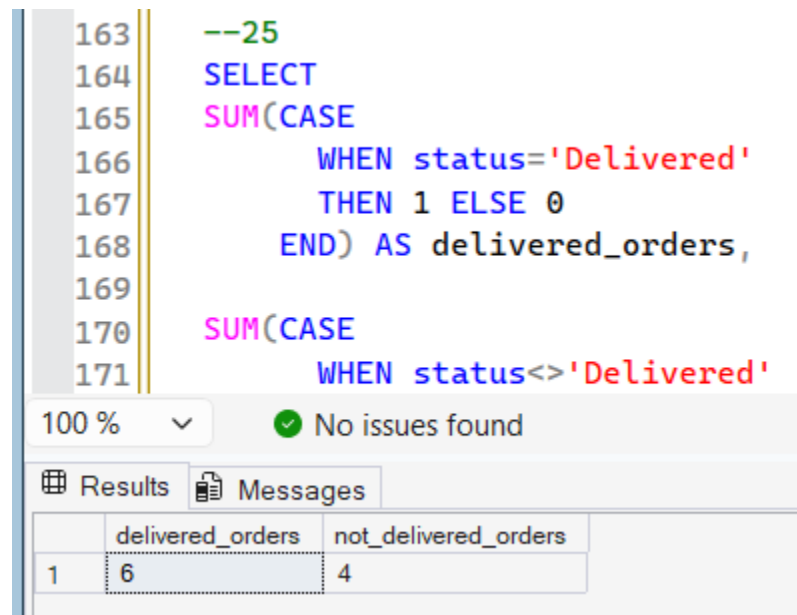
The results pane shows a table with 8 rows and 4 columns: product_name, unit_price, and price_tier. The data is as follows:

	product_name	unit_price	price_tier
1	Wireless Earbuds	1499.00	Mid-Range
2	Cotton T-Shirt	799.00	Budget
3	Smart Watch	2999.00	Mid-Range
4	Running Shoes	4599.00	Premium
5	Bluetooth Speaker	3499.00	Premium
6	Bedsheet Set	1299.00	Mid-Range
7	Laptop Stand	899.00	Budget
8	Cushion Covers (Set)	599.00	Budget

25.Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

```
SELECT
SUM(CASE
    WHEN status='Delivered'
    THEN 1 ELSE 0
END) AS delivered_orders,

SUM(CASE
    WHEN status<>'Delivered'
    THEN 1 ELSE 0
END) AS not_delivered_orders
FROM orders;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
--25
SELECT
SUM(CASE
    WHEN status='Delivered'
    THEN 1 ELSE 0
END) AS delivered_orders,

SUM(CASE
    WHEN status<>'Delivered'
    THEN 1 ELSE 0
END) AS not_delivered_orders
FROM orders;
```

The results pane shows a table with two columns: `delivered_orders` and `not_delivered_orders`. The table has one row with the values 6 and 4 respectively.

	delivered_orders	not_delivered_orders
1	6	4

26.Explain each letter of ACID:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

Give a real-world example (e.g., bank transfer) showing why each property is important.

A — Atomicity

Either all operations happen or none.

Example:

Bank transfer ₹1000 from A to B.

If money deducted from A but not added to B, transaction is rolled back.

C — Consistency

Database remains valid before and after transaction.

Example:

Account balances must always follow business rules.

I — Isolation

Multiple transactions do not interfere.

Example:

Two users withdrawing simultaneously should not create incorrect balances.

D — Durability

Once committed, data is permanent.

Example:

After successful bank transfer, data remains saved even if power fails.

27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)

2. Insert two order items for that order

3. Update the stock_qty of the purchased products

4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

BEGIN TRY

BEGIN TRANSACTION;

-- 1. Insert New Order

INSERT INTO orders

(order_id, customer_id, order_date, status, total_amount)

VALUES

(1011, 102, GETDATE(), 'Pending', 1598.00);

-- 2. Insert First Order Item

INSERT INTO order_items

(item_id, order_id, product_id, quantity, unit_price, discount_pct)

VALUES

(5016, 1011, 202, 1, 799.00, 0);

-- 3. Insert Second Order Item

INSERT INTO order_items

(item_id, order_id, product_id, quantity, unit_price, discount_pct)

VALUES

(5017, 1011, 208, 1, 799.00, 0);

-- 4. Update Product Stock

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 202;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 208;

-- Commit if all statements succeed

COMMIT TRANSACTION;

PRINT 'Transaction Completed Successfully';

END TRY

BEGIN CATCH

-- Rollback if any statement fails

ROLLBACK TRANSACTION;

PRINT 'Transaction Failed - All Changes Rolled Back';

END CATCH;

```
178 --27
179 BEGIN TRY
180     BEGIN TRANSACTION;
181
182     -- 1. Insert New Order
183     INSERT INTO orders
184     (order_id, customer_id, order_date, status, total_amount)
185     VALUES
186     (1011, 102, GETDATE(), 'Pending', 1598.00);
187
188     -- 2. Insert First Order Item
189     INSERT INTO order_items
190     (item_id, order_id, product_id, quantity, unit_price, discount_pct)
191     VALUES
192     (5016, 1011, 202, 1, 799.00, 0);
193
```

91 %

✓ No issues found

Messages

(0 rows affected)
Transaction Failed - All Changes Rolled Back
Completion time: 2026-05-31T17:38:42.8889155+05:30